

Parte de regresión (autoMPG6)

Andrei Kostin

Descargamos las bibliotecas necesarias:

```
library(readr)
library(tidyverse)
```

```
## -- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
## v dplyr      1.1.4      v purrr      1.1.0
## v forcats    1.0.1      v stringr   1.5.2
## v ggplot2    4.0.0      v tibble    3.3.0
## v lubridate  1.9.4      v tidyr     1.3.1
## -- Conflicts ----- tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()    masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```
library(dplyr)
library(tidyr)
library(ggplot2)
library(MASS)
```

```
##
##           : 'MASS'
##
##           'package:dplyr':
##
## select
```

```
library(kknn)
```

Leemos el dataset objetivo y nombramos las columnas:

```
df <- read.csv("D:/UGR/Introduction to DS/Tarea final/autoMPG6/autoMPG6.dat",
               comment.char="@", header = FALSE)

names(df) <- c("Displacement", "Horse_power", "Weight",
               "Acceleration", "Model_year", "Mpg")
head(df)
```

```
##   Displacement Horse_power Weight Acceleration Model_year  Mpg
## 1           91          70  1955          20.5         71 26.0
## 2          232         100  2789          15.0         73 18.0
```

## 3	350	145	4055	12.0	76 13.0
## 4	318	140	4080	13.7	78 17.5
## 5	113	95	2372	15.0	70 24.0
## 6	97	60	1834	19.0	71 27.0

1. Regresión lineal

Análisis de todas las variables del conjunto de datos según sus indicadores de calidad:

```
fit_weight <- lm(Mpg ~ Weight, data = df)
fit_hp <- lm(Mpg ~ Horse_power, data = df)
fit_disp <- lm(Mpg ~ Displacement, data = df)
fit_acc <- lm(Mpg ~ Acceleration, data = df)
fit_year <- lm(Mpg ~ Model_year, data = df)

simple_stats <- data.frame(var = c("Weight", "Horse_power", "Displacement",
                                   "Acceleration", "Model_year"),
                          r2 = c(summary(fit_weight)$r.squared,
                                   summary(fit_hp)$r.squared,
                                   summary(fit_disp)$r.squared,
                                   summary(fit_acc)$r.squared,
                                   summary(fit_year)$r.squared),
                          r2adj = c(summary(fit_weight)$adj.r.squared,
                                      summary(fit_hp)$adj.r.squared,
                                      summary(fit_disp)$adj.r.squared,
                                      summary(fit_acc)$adj.r.squared,
                                      summary(fit_year)$adj.r.squared),
                          rmse = c(sqrt(sum(fit_weight$residuals^2)/length(fit_weight$residuals)),
                                      sqrt(sum(fit_hp$residuals^2)/length(fit_hp$residuals)),
                                      sqrt(sum(fit_disp$residuals^2)/length(fit_disp$residuals)),
                                      sqrt(sum(fit_acc$residuals^2)/length(fit_acc$residuals)),
                                      sqrt(sum(fit_year$residuals^2)/length(fit_year$residuals)))
)

simple_stats
```

##	var	r2	r2adj	rmse
## 1	Weight	0.6926304	0.6918423	4.321645
## 2	Horse_power	0.6059483	0.6049379	4.893226
## 3	Displacement	0.6482294	0.6473274	4.623261
## 4	Acceleration	0.1792071	0.1771025	7.062126
## 5	Model_year	0.3370278	0.3353279	6.346968

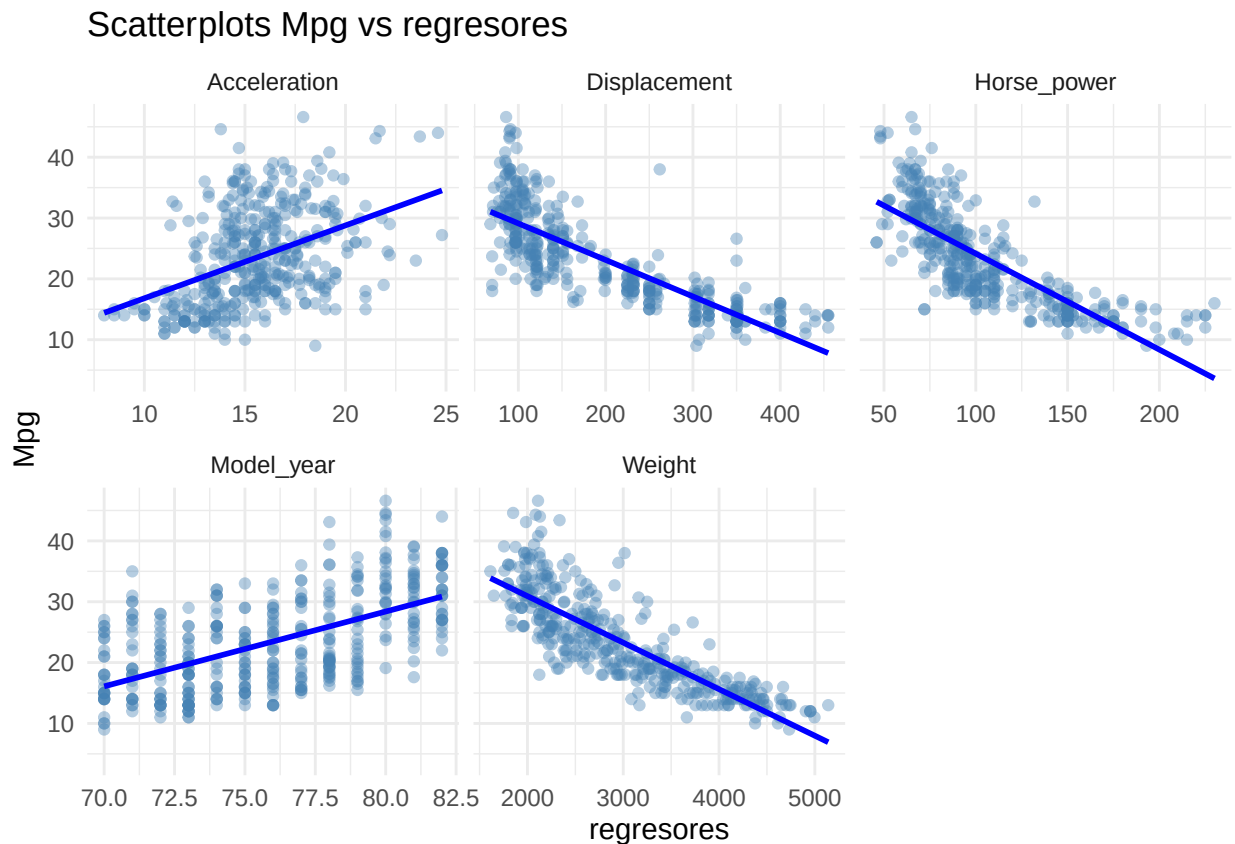
Los resultados de las cinco regresiones lineales simples muestran que *Weight* es el regresor más informativo para explicar *Mpg* debido a que tiene el R2 ajustado de ~0.69 y la menor raíz del error cuadrático medio (RMSE) igual a ~4.32. Las variables *Displacement* y *Horse_power* asimismo presentan relaciones fuertes con *Mpg* con R2 ajustado de 0.65 y 0.60, respectivamente. Por lo tanto, el modelo *Mpg ~ Weight* se selecciona como el mejor modelo de regresión lineal simple para este conjunto de datos.

Igualmente, abajo está la visualización de los regresores con los diagramas de dispersión:

```
df_long <- df %>% pivot_longer(cols = -Mpg, names_to = "regr", values_to = "values")

ggplot(df_long, aes(x = values, y = Mpg)) +
  geom_point(alpha = .4, color = "steelblue") +
  geom_smooth(method = "lm", se = FALSE, color = "blue") +
  facet_wrap(~ regr, scales = "free_x") +
  labs(title = "Scatterplots Mpg vs regresores", x = "regresores") +
  theme_minimal()
```

```
## `geom_smooth()` using formula = 'y ~ x'
```



En los gráficos de *Weight*, *Displacement* y *Horse_power* se ve una relación lineal negativa fuerte: a medida que aumentan estas variables, el valor de *Mpg* disminuye de forma casi lineal. Por el contrario, las relaciones de *Mpg* con las variables *Acceleration* y, especialmente, *Model_year* son más dispersas, así que menos explicativos para regresión.

2. Regresión lineal múltiple

Obtención del modelo de regresión con todas las variables:

```
fit_all = lm(Mpg~., data = df)
summary(fit_all)
```

```
##
```

```
## Call:
## lm(formula = Mpg ~ ., data = df)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -8.5211 -2.3920 -0.1036  2.0312 14.2874
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept) -1.544e+01  4.677e+00  -3.300  0.00106 **
## Displacement  2.782e-03  5.462e-03   0.509  0.61082
## Horse_power   1.020e-03  1.376e-02   0.074  0.94095
## Weight       -6.874e-03  6.653e-04 -10.333 < 2e-16 ***
## Acceleration  9.032e-02  1.019e-01   0.886  0.37599
## Model_year    7.541e-01  5.261e-02  14.334 < 2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 3.435 on 386 degrees of freedom
## Multiple R-squared:  0.8088, Adjusted R-squared:  0.8063
## F-statistic: 326.5 on 5 and 386 DF,  p-value: < 2.2e-16
```

De acuerdo con resultados de las medidas de calidad, *Weight* y *Model_year* muestran coeficientes altamente significativos, mientras que *Displacement*, *Horse_power* y *Acceleration* tienen p-valores elevados, lo que sugiere que proporcionan la información adicional sin importancia. Por lo tanto, en el siguiente paso eliminamos una variable por una en el orden de relevancia según p-value:

```
fit1 = lm(Mpg~.-Horse_power, data = df)
summary(fit1)
```

```
##
## Call:
## lm(formula = Mpg ~ . - Horse_power, data = df)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -8.5182 -2.3948 -0.1085  2.0405 14.2908
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept) -1.527e+01  4.106e+00  -3.719  0.000229 ***
## Displacement  2.874e-03  5.310e-03   0.541  0.588651
## Weight       -6.852e-03  5.967e-04 -11.483 < 2e-16 ***
## Acceleration  8.555e-02  7.885e-02   1.085  0.278595
## Model_year    7.532e-01  5.118e-02  14.717 < 2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 3.431 on 387 degrees of freedom
## Multiple R-squared:  0.8088, Adjusted R-squared:  0.8068
## F-statistic: 409.2 on 4 and 387 DF,  p-value: < 2.2e-16
```

```
fit2 = lm(Mpg~.-Horse_power-Displacement, data = df)
summary(fit2)
```

```
##
## Call:
## lm(formula = Mpg ~ . - Horse_power - Displacement, data = df)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -8.6749 -2.3528 -0.1082  2.0168 14.3022
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept) -14.936555   4.055512  -3.683 0.000263 ***
## Weight      -0.006554   0.000230 -28.502 < 2e-16 ***
## Acceleration  0.066359   0.070361   0.943 0.346204
## Model_year    0.748446   0.050366  14.860 < 2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 3.428 on 388 degrees of freedom
## Multiple R-squared:  0.8086, Adjusted R-squared:  0.8071
## F-statistic: 546.5 on 3 and 388 DF,  p-value: < 2.2e-16
```

```
fit3 = lm(Mpg~.-Horse_power-Displacement-Acceleration, data = df)
summary(fit3)
```

```
##
## Call:
## lm(formula = Mpg ~ . - Horse_power - Displacement - Acceleration,
##      data = df)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -8.8505 -2.3014 -0.1167  2.0367 14.3555
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept) -1.435e+01  4.007e+00  -3.581 0.000386 ***
## Weight      -6.632e-03  2.146e-04 -30.911 < 2e-16 ***
## Model_year   7.573e-01  4.947e-02  15.308 < 2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 3.427 on 389 degrees of freedom
## Multiple R-squared:  0.8082, Adjusted R-squared:  0.8072
## F-statistic: 819.5 on 2 and 389 DF,  p-value: < 2.2e-16
```

De conformidad con resultados obtenidos, se observa la disminución coherente del error residual. El modelo reducido que únicamente utiliza *Weight* y *Model_year* alcanza un R² ajustado de 0.807 y el menor error estándar residual. Estas variables están presentadas en las hipótesis de la parte EDA que establecen que a mayor peso disminuye *Mpg*, mientras que los modelos más recientes presentan un consumo más eficiente.

Comprobación de interacción de las variables más relevantes según análisis anterior:

```
fit4 = lm(Mpg~Weight*Model_year, data = df)
summary(fit4)
```

```
##
## Call:
## lm(formula = Mpg ~ Weight * Model_year, data = df)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -8.0397 -1.9956 -0.0983  1.6525 12.9896
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)   -1.105e+02  1.295e+01  -8.531 3.30e-16 ***
## Weight         2.755e-02  4.413e-03   6.242 1.14e-09 ***
## Model_year     2.040e+00  1.718e-01  11.876 < 2e-16 ***
## Weight:Model_year -4.579e-04  5.907e-05  -7.752 8.02e-14 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 3.193 on 388 degrees of freedom
## Multiple R-squared:  0.8339, Adjusted R-squared:  0.8326
## F-statistic: 649.3 on 3 and 388 DF,  p-value: < 2.2e-16
```

En este caso las medidas de calidad mejoran fuertemente. Esto indica que el efecto del peso sobre *Mpg* depende del año del modelo.

Adición de no linealidad:

```
fit5 = lm(Mpg~Weight + I(Weight^2) + Model_year, data = df)
summary(fit5)
```

```
##
## Call:
## lm(formula = Mpg ~ Weight + I(Weight^2) + Model_year, data = df)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -9.4561 -1.7083 -0.1726  1.5192 13.1789
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)   2.121e+00  3.879e+00   0.547   0.585
## Weight        -2.155e-02  1.441e-03 -14.955 <2e-16 ***
## I(Weight^2)    2.348e-06  2.248e-07  10.443 <2e-16 ***
## Model_year     8.289e-01  4.430e-02  18.712 <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 3.032 on 388 degrees of freedom
## Multiple R-squared:  0.8503, Adjusted R-squared:  0.8491
## F-statistic: 734.4 on 3 and 388 DF,  p-value: < 2.2e-16
```

Después de incluir un término cuadrático en *Weight*, finalmente el resultado mejora considerablemente en comparación con todos los análisis anteriores.

Combinación de los dos últimos pasos:

```
fit6 = lm(Mpg~Weight*Model_year +
          Weight + I(Weight^2) +
          Model_year + I(Model_year^2), data = df)
summary(fit6)

##
## Call:
## lm(formula = Mpg ~ Weight * Model_year + Weight + I(Weight^2) +
##     Model_year + I(Model_year^2), data = df)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -8.7982 -1.7094  0.0139  1.3190 12.8078
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)    2.516e+02  8.244e+01   3.052 0.002432 **
## Weight        -8.217e-03  5.757e-03  -1.427 0.154302
## Model_year    -6.242e+00  2.074e+00  -3.009 0.002789 **
## I(Weight^2)     2.044e-06  2.434e-07   8.396 8.88e-16 ***
## I(Model_year^2)  4.928e-02  1.313e-02   3.753 0.000201 ***
## Weight:Model_year -1.517e-04  6.556e-05  -2.314 0.021166 *
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 2.923 on 386 degrees of freedom
## Multiple R-squared:  0.8615, Adjusted R-squared:  0.8597
## F-statistic: 480.3 on 5 and 386 DF,  p-value: < 2.2e-16
```

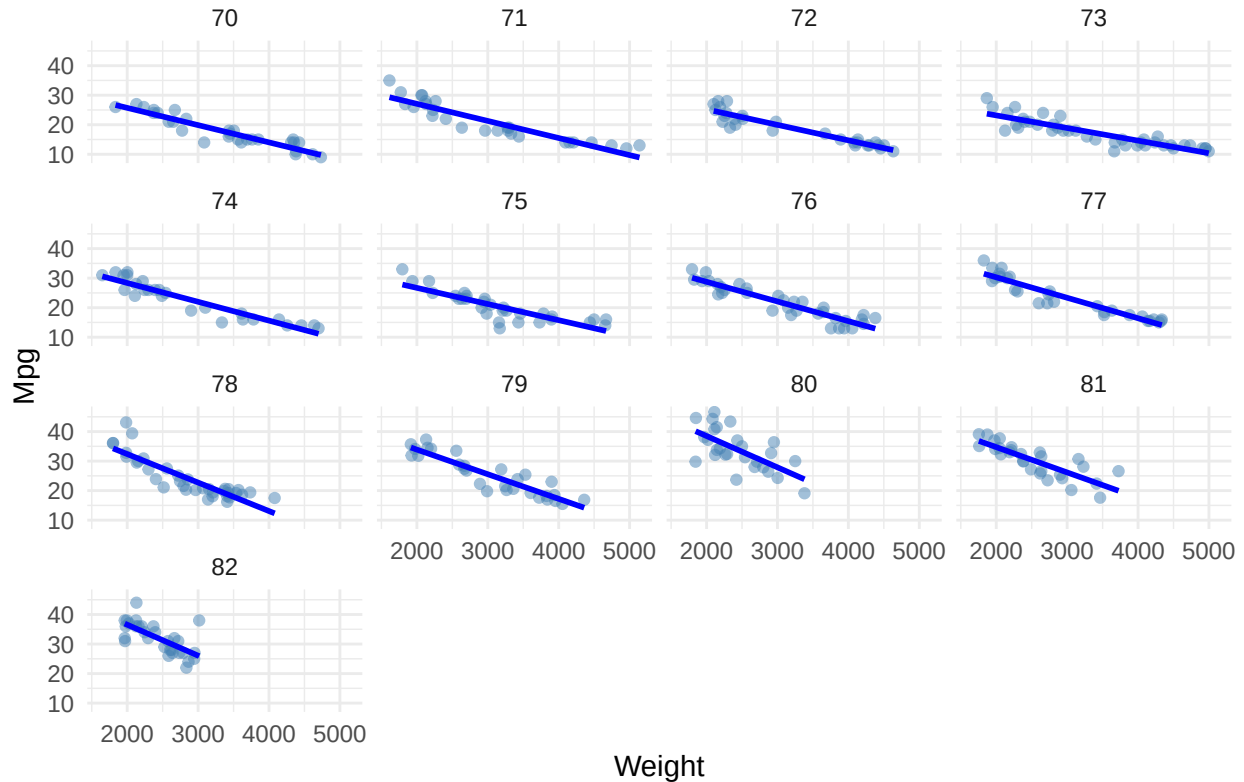
Al final conseguimos los mejores resultados combinando la interacción y los coeficientes cuadráticos de ambas variables *Weight* y *Model_year*. Esto confirma la presencia de relaciones no lineales y de un efecto del peso que depende del año del modelo.

Representamos la relación entre *Mpg* y *Weight* por cada año:

```
ggplot(df, aes(x = Weight, y = Mpg)) +
  geom_point(alpha = .5, color = "steelblue") +
  geom_smooth(method = "lm", se = FALSE, color = "blue") +
  facet_wrap(~ Model_year) +
  labs(title = "Relación Mpg-Weight por grupos de años") +
  theme_minimal()
```

```
## `geom_smooth()` using formula = 'y ~ x'
```

Relación Mpg–Weight por grupos de años

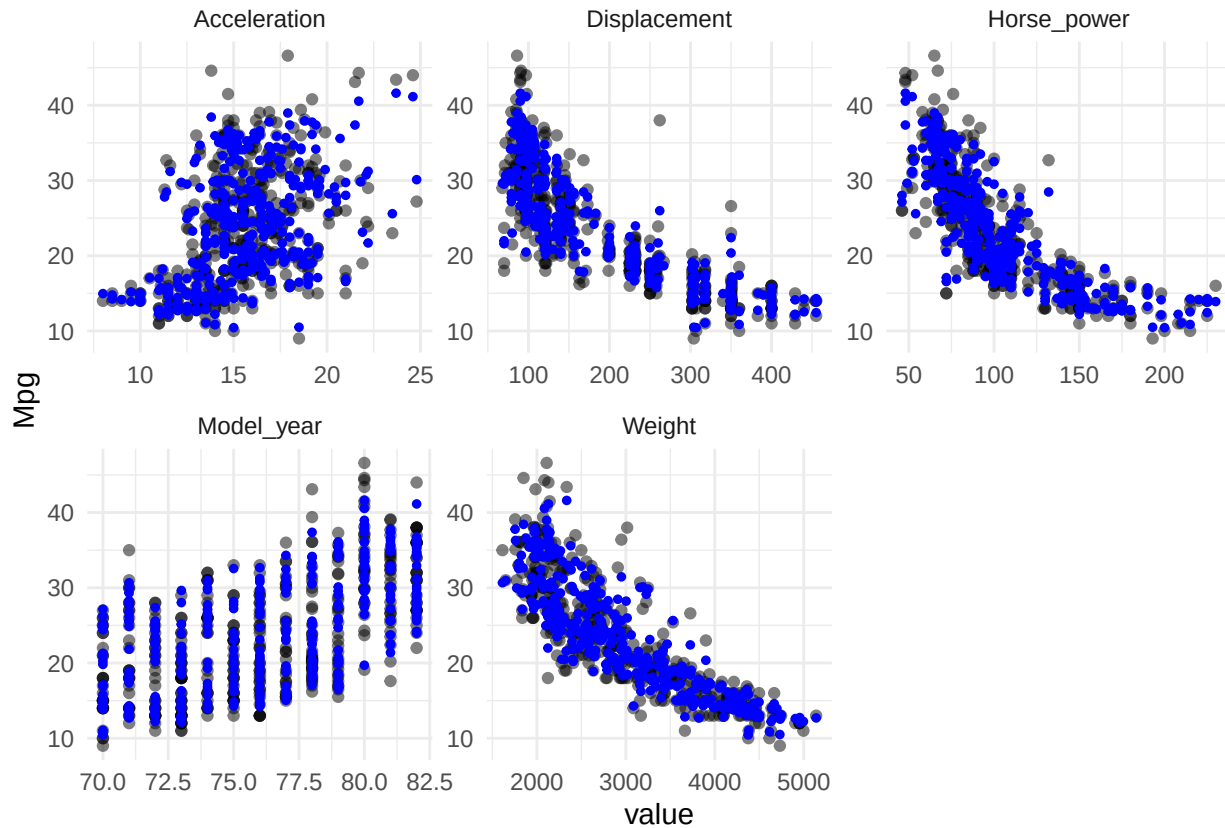


3. Aplicación el algoritmo k-NN para regresión no paramétrica

En los gráficos comparativos por variable se observa que las predicciones de **k-NN** siguen razonablemente bien las tendencias globales de los datos reales pero subestima los valores extremos:

```
fitknn_all <- kkn(Mpg ~ ., df, df, k=7, distance=2, kernel="optimal", scale=TRUE)

df_long <- df %>% mutate(fitted = fitknn_all$fitted.values) %>%
  pivot_longer(cols = -c(Mpg,fitted), names_to = "variable", values_to = "value")
df_long %>% ggplot(aes(x = value)) +
  geom_point(aes(y = Mpg), alpha = .5) +
  geom_point(aes(y = fitted), color = "blue", shape = 20, size = 1.7) +
  facet_wrap(~ variable, scale="free") + theme_minimal()
```

En primer lugar, leemos los archivos de train y test relativamente y aplicamos el modelo lineal según estos datos para obtener los scores del modelo:

```
# K-NN

name <- "autoMPG6"
run_lm_fold <- function(i, x, tt = "test") {
  file <- paste(x, "-5-", i, "tra.dat", sep=""); x_tra <- read.csv(file, comment.char="@", header=FALSE)
  file <- paste(x, "-5-", i, "tst.dat", sep=""); x_tst <- read.csv(file, comment.char="@", header=FALSE)
  names(x_tra) <- c("Displacement", "Horse_power", "Weight",
                   "Acceleration", "Model_year", "Mpg")
  names(x_tst) <- c("Displacement", "Horse_power", "Weight",
                   "Acceleration", "Model_year", "Mpg")
  if (tt == "train") {
    test <- x_tra
  }
  else {
    test <- x_tst
  }
  fitMulti=lm(Mpg~Weight*Model_year +
              Weight + I(Weight^2) +
              Model_year + I(Model_year^2), x_tra)
  yprime=predict(fitMulti,test)
  sum(abs(test$Mpg-yprime)^2)/length(yprime) ##MSE
}
lmMSEtrain<-mean(sapply(1:5,run_lm_fold,name,"train"))
lmMSEtest<-mean(sapply(1:5,run_lm_fold,name,"test"))
```

```
cat("LM train score: ", lmMSEtrain, "\n")
```

```
## LM train score: 8.396414
```

```
cat("LM test score: ", lmMSEtest, "\n")
```

```
## LM test score: 8.564953
```

En segundo lugar, ejecutamos lo mismo, pero con el modelo k-NN:

```
library(kknn)
name <- "autoMPG6"
run_knn_fold <- function(i, x, tt = "test") {
  file <- paste(x, "-5-", i, "tra.dat", sep=""); x_tra <- read.csv(file, comment.char="@", header=FALSE)
  file <- paste(x, "-5-", i, "tst.dat", sep=""); x_tst <- read.csv(file, comment.char="@", header=FALSE)
  names(x_tra) <- c("Displacement", "Horse_power", "Weight",
                   "Acceleration", "Model_year", "Mpg")
  names(x_tst) <- c("Displacement", "Horse_power", "Weight",
                   "Acceleration", "Model_year", "Mpg")
  if (tt == "train") {
    test <- x_tra
  }
  else {
    test <- x_tst
  }
  fitMulti=kknn(Mpg~Weight*Model_year +
                Weight + I(Weight^2) +
                Model_year + I(Model_year^2), train = x_tra, test = test)
  yprime=fitMulti$fitted.values
  sum(abs(test$Mpg-yprime)^2)/length(yprime) ##MSE
}
knnMSEtrain<-mean(sapply(1:5,run_knn_fold,name,"train"))
knnMSEtest<-mean(sapply(1:5,run_knn_fold,name,"test"))
cat("K-NN train score: ", knnMSEtrain, "\n")
```

```
## K-NN train score: 3.985457
```

```
cat("K-NN test score: ", knnMSEtest, "\n")
```

```
## K-NN test score: 8.758878
```

Los resultados del train y test del **modelo lineal** y **k-NN** relativamente:

```
data.frame(tipo_de_modelo = c("LM", "k-NN"),
           train_score = c(lmMSEtrain, knnMSEtrain),
           test_score = c(lmMSEtest, knnMSEtest))
```

```
##  tipo_de_modelo train_score test_score
## 1              LM      8.396414  8.564953
## 2             k-NN      3.985457  8.758878
```

El modelo k-NN obtiene un error mucho menor en entrenamiento que la regresión lineal múltiple, lo que indica una mayor capacidad de ajuste, pero en las particiones de test su resultado de RMSE (8.76) es ligeramente superior al del modelo lineal (8.56). Por lo tanto, k-NN no mejora el rendimiento predictivo sobre datos nuevos y muestra una ligera tendencia al sobreajuste frente al modelo de regresión lineal.

4. Comparación de los resultados de los algoritmos de regresión múltiple

Leemos los archivos .CSV y cambiamos los resultados de los modelos LM y k-NN para el conjunto de datos autoMPG6:

```
# leer train file
res_train <- read.csv("regr_train_alumnos.csv")

# cambiamos los resultados de dos modelos para autoMPG6
idx <- which(rownames(res_train) == "autoMPG6")
res_train[res_train[,1] == "autoMPG6", "out_train_lm"] <- lmMSEtrain
res_train[res_train[,1] == "autoMPG6", "out_train_kknn"] <- knnMSEtrain
write.csv(res_train, "regr_train_alumnos.csv", row.names = FALSE)

tablatra <- as.matrix(res_train[, c("out_train_lm", "out_train_kknn", "out_train_m5p")])
colnames(tablatra) <- c("LM", "KNN", "M5")
rownames(tablatra) <- res_train[,1]

# igual con el test file
res_test <- read.csv("regr_test_alumnos.csv")

res_test[res_test[,1] == "autoMPG6", "out_test_lm"] <- lmMSEtest
res_test[res_test[,1] == "autoMPG6", "out_test_kknn"] <- knnMSEtest
write.csv(res_test, "regr_test_alumnos.csv", row.names = FALSE)

tablatst <- as.matrix(res_test[, c("out_test_lm", "out_test_kknn", "out_test_m5p")])
colnames(tablatst) <- c("LM", "KNN", "M5")
rownames(tablatst) <- res_test[,1]
```

Para comparar los modelos de LM y k-NN sobre todos los conjuntos de datos, se ha aplicado el test de Wilcoxon:

```
# Calculamos las diferencia relativa LM - KNN
difs <- (tablatst[, "LM"] - tablatst[, "KNN"]) / tablatst[, "LM"]

# Transformamos los datos para el test Wilcoxon
# Y se crean dos columnas que contienen diferencias negativa y positiva
wilc_1_2 <- cbind(ifelse(difs < 0, abs(difs) + 0.1, 0 + 0.1), # LM es peor que KNN
                  ifelse(difs > 0, abs(difs) + 0.1, 0 + 0.1)) # LM es mejor que KNN
colnames(wilc_1_2) <- c("LM", "KNN")

# El test de Wilcoxon LM vs KNN bilateral y pareado
LMvsKNNtst <- wilcox.test(wilc_1_2[,1], wilc_1_2[,2],
                          alternative = "two.sided",
                          paired = TRUE)

# Guardamos el R+ y el p-valor de la primera ejecución
Rmas <- LMvsKNNtst$statistic
pvalue <- LMvsKNNtst$p.value
```

```

# Repetimos la prueba para obtener el valor estadístico R-
LMvsKNNtst <- wilcox.test(wilc_1_2[,2], wilc_1_2[,1],
                          alternative = "two.sided",
                          paired = TRUE)
Rmenos <- LMvsKNNtst$statistic

# Representación del resultado
data.frame(R_plus = Rmas, R_minus = Rmenos, p_value = pvalue)

```

```

##      R_plus R_minus  p_value
## V      84      87 0.9661179

```

El p-valor obtenido es 0.966, por lo que no se rechaza la hipótesis nula de igualdad de rendimiento. Es decir, LM y k-NN no muestran diferencias estadísticamente significativas.

Utilización del test de Friedman para comparar simultáneamente los tres algoritmos (LM, k-NN y M5’):

```

# Ejecutamos el test de Friedman
test_friedman <- friedman.test(as.matrix(tablatst))
test_friedman

```

```

##
## Friedman rank sum test
##
## data:  as.matrix(tablatst)
## Friedman chi-squared = 10.111, df = 2, p-value = 0.006374

```

```

# Preparación para la prueba de Wilcoxon por pares con corrección de Holm
tam <- dim(tablatst)
groups <- rep(1:tam[2], each = tam[1])

# Aplicación de post-hoc Holm
pairwise.wilcox.test(as.matrix(tablatst), groups, p.adjust = "holm", paired = TRUE)

```

```

##
## Pairwise comparisons using Wilcoxon signed rank exact test
##
## data:  as.matrix(tablatst) and groups
##
##      1      2
## 2 0.832 -
## 3 0.062 0.062
##
## P value adjustment method: holm

```

Este test sobre los errores de LM, k-NN y M5’ da un p-value de $0.006 < 0.05$, lo que indica la existencia de las diferencias significativas al menos entre un par de algoritmos. No obstante, el análisis post-hoc mediante comparaciones pareadas de Wilcoxon con corrección de Holm presentan p-values de 0.832 para el par LM/k-NN y de 0.062 para ambos pares LM/M5’ y k-NN/M5’, así que no se detectan diferencias estadísticamente significativas entre algunos pares.

Todos los resultados de error del dataset autoMPG6 según los tres algoritmos:

```
tablatst["autoMPG6", ]
```

```
##          LM          KNN          M5  
## 8.564953 8.758878 8.240000
```

De acuerdo con las medidas obtenidas el modelo de algoritmo M5' consigue un error un poco menor que los otros dos. Por lo tanto, los tres algoritmos presentan un rendimiento comparable en términos de error de predicción.